

I. Traitements de données

A. Importer des données depuis un tableur

Vous avez réalisé un jeu de mesures sur un tableur et avez obtenu un fichier type ".csv" ".xls" ou ".xlsx" à l'aide d'Excel ou Libre Office.

Comment l'importer rapidement sur Python ?

Nous allons pour cela utiliser les modules :

- `pandas` et `xlrd` pour l'import des données depuis le tableur
- `matplotlib.pyplot` pour les tracés
- `numpy` pour le traitement des tableaux

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import xlrd

#pour afficher les graphiques "en ligne"
import matplotlib
%matplotlib inline
matplotlib.rcParams['figure.figsize']=[8.0,8.0]

## pour avoir la main sur les graphiques (zoom, etc...)
#import matplotlib
#matplotlib.pyplot
#matplotlib.rcParams['figure.figsize']=[7.5,5]
```

Dans un premier temps, on récupère les données qui sont dans le tableur. Une bonne pratique est que chaque grandeur corresponde à une colonne du tableur, et que la première ligne corresponde au nom de la grandeur et à son unité. Par exemple, dans notre fichier `determinationg.xls`, les trois premières lignes sont :

Δt (s)	$u(\Delta t)$ (s)	h (m)	$u(h)$ (m)
0,25	0,05	0,5	0,02
0,42	0,05	0,75	0,02

On va importer les données du tableur dans Python sous la forme d'un `DataFrame`, appelé `mesdonnees` ci-dessous, à l'aide de la fonction `read_excel`

```
mesdonnees = pd.read_excel("determinationg.xls")
# si le chemin relatif ne fonctionne pas, vous pouvez indiquer le chemin absolu
# avec des // comme "C://Users//etc//monfichier.xlsx"
```

Il ne reste plus désormais qu'à récupérer chaque colonne du tableur sous forme d'un tableau `numpy` à l'aide de la méthode `values` suivante :

```
deltat=mesdonnees.values[:,0]
# les : indiquent que l'on prend toutes les lignes,
# le deuxième argument indique la colonne, la première étant repérée par 0
u_deltat=mesdonnees.values[:,1]
h=mesdonnees.values[:,2]
u_h=mesdonnees.values[:,3]
```

On peut vérifier que l'on a bien récupéré ce que l'on voulait :

```
deltat
```

```
Out[4]: array([0.25, 0.42, 0.5 , 0.53, 0.53, 0.58, 0.72, 0.6 , 0.8 , 0.7 , 0.7 ])
```

```
u_deltat
```

```
Out[5]: array([0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05, 0.05])
```

```
h
```

```
Out[6]: array([0.5 , 0.75, 1. , 1.25, 1.5 , 1.75, 2. , 2.25, 2.5 , 2.75, 3. ])
```

```
u_h
```

```
Out[7]: array([0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.02, 0.02])
```

B. Tracer une courbe avec les incertitudes

On s'intéresse à la durée d'une chute libre en fonction de la hauteur du lâcher. On a réalisé des expériences et on dispose de 4 tableaux `numpy` depuis un tableur :

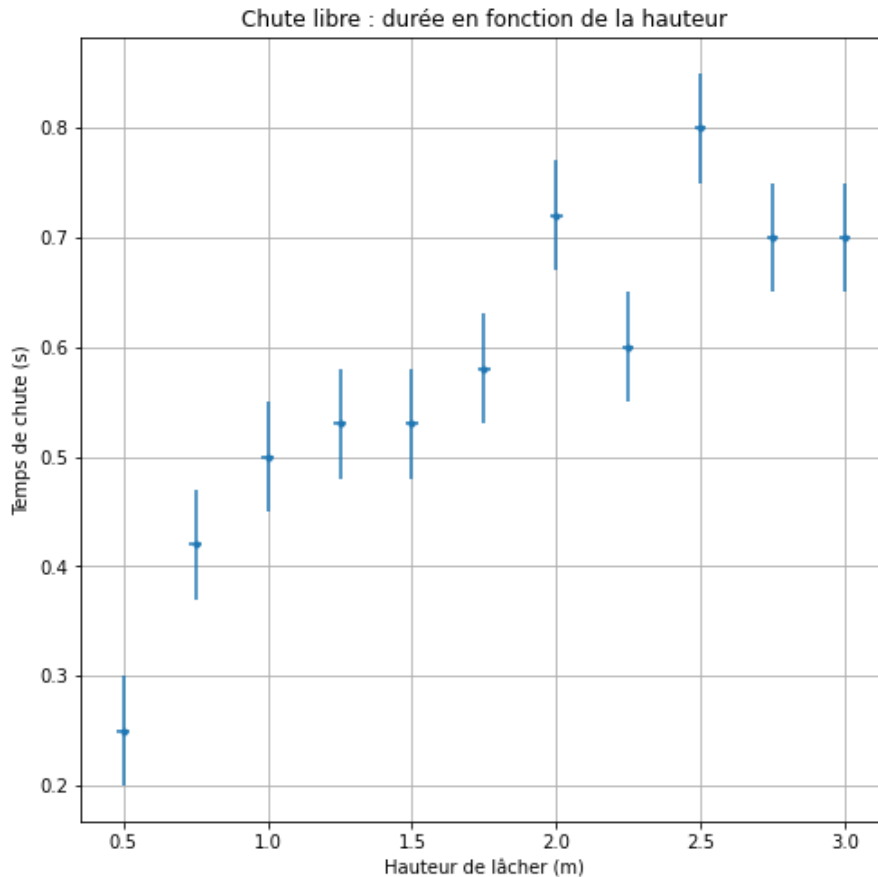
- `h` les hauteurs de lâché en m
- `u_h` les incertitudes type associées en m
- `deltat` les durées de chute en s
- `u_deltat` les incertitudes type associées en s

Comment tracer de manière élégante $\Delta t(h)$ en faisant apparaître les incertitudes type ?

Nous allons utiliser pour cela le module `matplotlib.pyplot`.

```
import matplotlib.pyplot as plt

plt.figure() #créons une nouvelle figure
plt.errorbar(h,deltat,xerr=u_h,yerr=u_deltat,fmt='.') #les incertitudes type sont
placées dans les arguments xerr et yerr
# fmt='.' permet de ne pas relier les points (chaque mesure est un point + barre
d'incertitude)
plt.grid(which='both') #on ajoute des grilles verticales et horizontales
plt.xlabel("Hauteur de lâcher (m)") #on nomme l'axe horizontal
plt.ylabel("Temps de chute (s)") #on nomme l'axe vertical
plt.title("Chute libre : durée en fonction de la hauteur") # titre (optionnel)
plt.show()
```



C. Réaliser une modélisation via Python

On s'intéresse à la durée d'une chute libre en fonction de la hauteur du lâcher. On a réalisé des expériences et on dispose de 4 tableaux `numpy` depuis un tableur :

- `h` les hauteurs de lâché en m
- `u_h` les incertitudes type associées en m
- `deltat` les durées de chute en s
- `u_deltat` les incertitudes type associées en s

Dans la partie précédente, on a fait le tracé de Δt en fonction de h . On sait que l'expression théorique associée est, en notant g le champ de pesanteur :

$$\Delta t = \sqrt{\frac{2h}{g}}$$

On va demander à Python de faire de trouver la courbe dont l'équation est la précédente, et qui passe au mieux par les points de mesure. Il en déduira alors la valeur de g .

Voici la démarche générale (ne pas compiler) :

```
from scipy.optimize import curve_fit

def f(x,a,b,c,d): # mettre la variable puis les paramètres à estimer en argument.
    return a*x**3+b*x**2+c*x+d # mettre ici l'expression à approcher

x = [...] # rentrer ici la liste des mesures de x
y = [...] # rentrer ici la liste des mesures de y associées

p_estimation,p_err=curve_fit(f, x, y) #p_estimation est la liste des paramètres
estimés
p_type=np.sqrt(np.diag(p_err)) #p_type est la liste des incertitudes type sur
l'estimation des paramètres
```

```
## ne pas oublier de finir par un tracé superposant les points de mesure et la courbe
théorique estimée pour valider
## l'estimation.

## si la modélisation ne converge pas, on peut « aider » curve_fit, en lui donnant une
première estimation des
## paramètres dans un argument p0=[aestime,bestime,cestime,destime]
```

Mettons-la en application sur notre exemple :

```
from scipy.optimize import curve_fit # c'est la fonction qui réalise l'ajustement
from math import sqrt

#tout d'abord on définit la fonction à modéliser

def f(x,g):
    return np.sqrt(2*x/g)

x=h
y=deltat

p_estimation,p_err=curve_fit(f, x, y) #p_estimation est la liste des paramètres
estimés
p_type=np.sqrt(np.diag(p_err)) #p_type est la liste des incertitudes type sur
l'estimation des paramètres
```

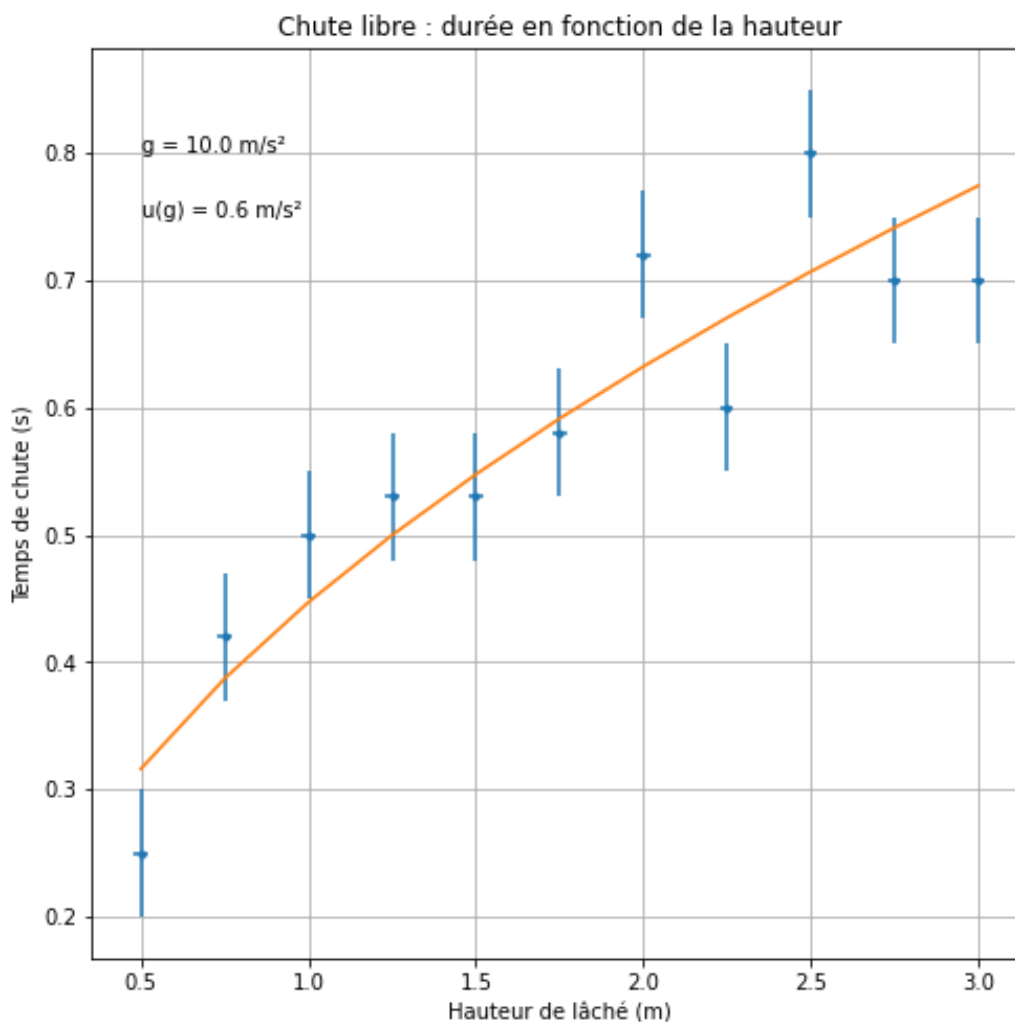
Vérifions la valeur de g , ainsi que le tracé de la courbe modèle :

```
g=p_estimation[0]
u_g=p_type[0]

#le tracé de nos mesures (cf chapitre précédent)
plt.figure()
plt.errorbar(h,deltat,xerr=u_h,yerr=u_deltat,fmt='.')
plt.grid(which='both') #on ajoute des grilles verticales et horizontales
plt.xlabel("Hauteur de lâché (m)") #on nomme l'axe horizontal
plt.ylabel("Temps de chute (s)") #on nomme l'axe vertical
plt.title("Chute libre : durée en fonction de la hauteur") # titre (optionnel)

#le tracé de la modélisation
plt.plot(h,f(h,g))
plt.text(0.5,0.8,"g = "+str(g)[0:4]+" m/s²")
plt.text(0.5,0.75,"u(g) = "+str(u_g)[0:3]+" m/s²")

plt.show()
```



Avant d'accepter les valeurs déterminées par la modélisation on vérifie que :

- les **points de mesure sont aléatoirement répartis** autour de la courbe de modélisation
- la **courbe** est toujours "**à proximité**" des **points** (pas plus de 2 écarts-types)

Dans ce cas, on peut faire confiance au résultat.

Exemple : ici $g = 10.0 \text{ m.s}^{-2}$; $u(g) = 0.6 \text{ m.s}^{-2}$. On ne garde qu'un seul chiffre significatif pour cette méthode de détermination de l'incertitude-type.

Il faudrait une méthode de Monte Carlo pour en avoir deux, c'est ce que l'on va traiter maintenant.

II. Traitements des incertitudes

A. Propagation des incertitudes par la méthode de Monte-Carlo

On dispose :

- d'une grandeur x dont la valeur mesurée est x_m associée à son incertitude-type $u(x)$
- d'une grandeur y dont la valeur mesurée est y_m associée à son incertitude-type $u(y)$

On veut calculer une grandeur $z = f(x, y)$ et estimer l'incertitude-type sur cette grandeur par un algorithme de Monte-Carlo.

La démarche va se faire en trois temps :

1. On va générer aléatoirement des tableaux de valeurs de x et y , de distribution uniforme, et cohérentes avec les valeurs des incertitudes-type (c'est à dire que, pour x , les valeurs sont uniformément réparties dans l'intervalle $[x - \sqrt{3}u(x); x + \sqrt{3}u(x)]$. Ces listes sont appelées x et y dans la suite.
2. Pour chaque couple $x[i], y[i]$, on calcule la valeur $z[i]=f(x[i],y[i])$
3. L'incertitude-type sur z est l'écart-type du tableau de valeurs de z qui vient d'être calculé.

```
import random as rd
import numpy as np
from math import sqrt
import matplotlib.pyplot as plt

def uniforme(m,u,N): # va renvoyer un tableau de N nombres aléatoire correspondant à
une distribution uniforme
# centrée sur m et d'incertitude-type u
    x=[]
    a=m-sqrt(3)*u
    b=m+sqrt(3)*u
    for i in range(N):
        x.append(rd.uniform(a,b)) # la fonction rd.uniform renvoie un nombre aléatoire
entre a et b (densité uniforme)
    return np.array(x)

## génération des tableaux x et y

xm= 5.3 # on rentre ici les données issues de l'expérience (grandeur mesurée et
incertitude-type)
u_x=0.7

ym= 2.1
u_y=0.4

x=uniforme(xm,u_x,10000)
y=uniforme(ym,u_y,10000)

## définition de la fonction f

def f(x,y):
    return 3*x**2*y

## calcul de la liste des z

z=f(x,y)

## tracé des histogrammes et détermination de l'incertitude type sur z

plt.figure()

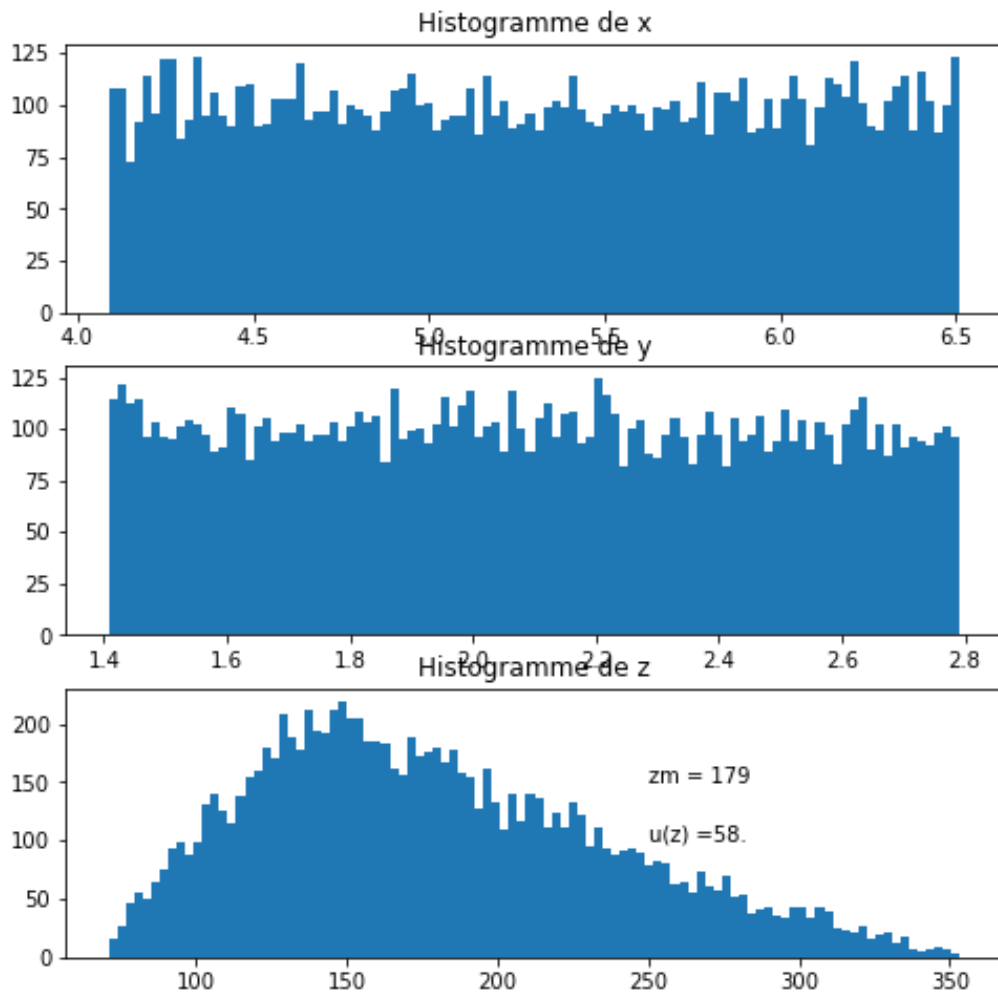
plt.subplot(311)
plt.hist(x,bins=100)
plt.title("Histogramme de x")

plt.subplot(312)
plt.hist(y,bins=100)
plt.title("Histogramme de y")

plt.subplot(313)
plt.hist(z,bins=100)
plt.title("Histogramme de z")

zm=np.mean(z)
um=np.std(z)
```

```
plt.text(250,150, "zm = "+str(zm) [0:3])
plt.text(250,100, "u(z) =" +str(um) [0:3])
plt.show()
```



B. Evaluer l'incertitude-type sur un paramètre inconnu avec la méthode de Monte-Carlo

i. Traitement à partir d'un tableau de mesure

A l'issue d'une modélisation/ajustement d'une courbe avec `curve_fit`, on peut obtenir la valeur d'un paramètre inconnu (g dans l'exemple plus haut), associé à une incertitude-type $u(g)$. Cependant, nous ne savons pas comment est déterminée cette incertitude-type et on aimerait la déterminer nous-même par un algorithme de type Monte-Carlo.

Comment estimer l'incertitude-type sur les paramètres d'une fonction ff telle que $y = f(x)$ où y et x sont des grandeurs mesurées ?

On dispose :

- d'un tableau `lx` des valeurs de x mesurées, de longueur n
- d'un tableau `lux` des valeurs de $u(x)$ mesurées, de longueur n
- d'un tableau `ly` des valeurs de y mesurées, de longueur n
- d'un tableau `lux` des valeurs de $u(y)$ mesurées, de longueur n

L'algorithme consiste en :

Faire NN fois :

1) Génération des tableaux simulés `lx_simu` et `ly_simu` avec une distribution aléatoire :

$\forall i$ $lx_simu[i]$ est une valeur aléatoire tirée uniformément dans $[lx[i] - \sqrt{3} \times lux[i]; lx[i] + \sqrt{3} \times lux[i]]$

On fait de même pour le tableau des y .

2) Estimation des paramètres inconnus à l'aide de `curve_fit` à partir de `lx_simu` et `ly_simu`.

3) Stockage des paramètres estimés dans une liste qui grandit à chaque itération.

Après les N étapes : Observer l'histogramme du(des) paramètre(s) estimé(s) et calculer l'écart-type associé qui donne l'incertitude type sur le paramètre.

```
## on reprend la détermination de g commencée plus haut

# on commence par ré-importer les données

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit # c'est la fonction qui réalise l'ajustement
from math import sqrt
import random as rd

mesdonnees = pd.read_excel("determinationg.xls")
deltat=mesdonnees.values[:,0]
u_deltat=mesdonnees.values[:,1]
h=mesdonnees.values[:,2]
u_h=mesdonnees.values[:,3]

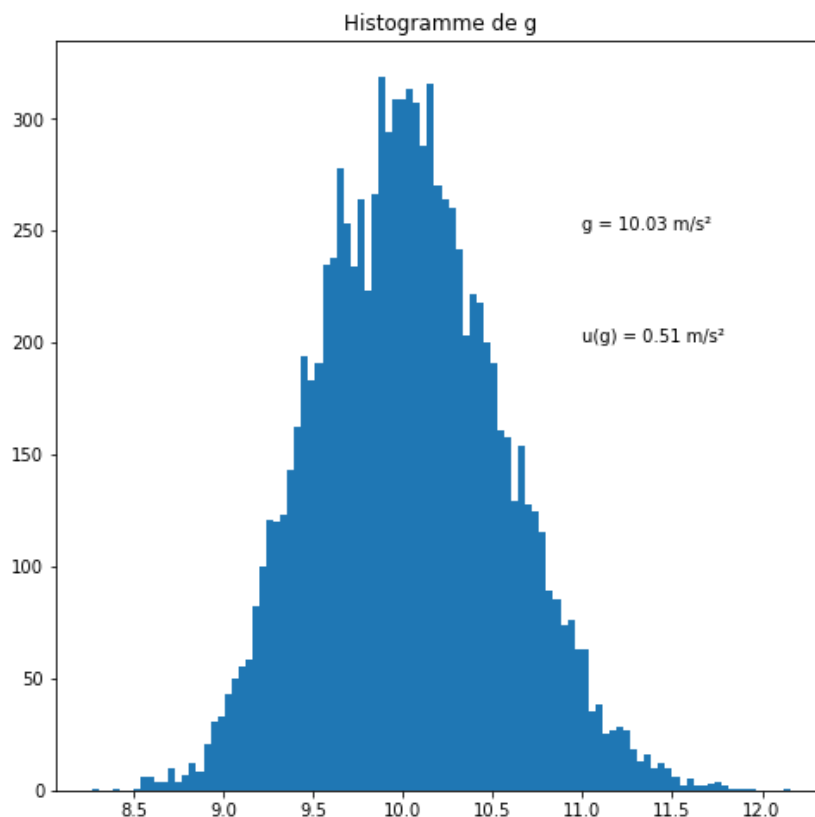
# on redéfinit la fonction de modélisation

def f(x,g):
    return np.sqrt(2*x/g)

# on lance l'algorithme de Monte Carlo
liste_g=[]
N=10000
deltat_simu=np.zeros((len(deltat)))
h_simu=np.zeros((len(deltat)))
for k in range(N):
    # étape 1 : on génère les tableaux simulés
    for i in range(len(deltat)):
        deltat_simu[i]=rd.uniform(deltat[i]-
sqrt(3)*u_deltat[i],deltat[i]+sqrt(3)*u_deltat[i])
        h_simu[i]=rd.uniform(h[i]-sqrt(3)*u_h[i],h[i]+sqrt(3)*u_h[i])
    # étape 2 : on estime les paramètres inconnus
    x=h_simu
    y=deltat_simu
    p_estimation,p_err=curve_fit(f, x, y)
    # étape 3 : on remplit la liste des g estimés
    liste_g.append(p_estimation[0])

gm=np.mean(np.array(liste_g))
ug=np.std(np.array(liste_g))

#tracés
plt.figure()
plt.title("Histogramme de g")
plt.hist(liste_g,bins=100)
plt.text(11,250,"g = "+str(gm)[0:5]+" m/s²")
plt.text(11,200,"u(g) = "+str(ug)[0:4]+" m/s²")
plt.show()
```



ii. Traitement à partir d'une seule mesure

On peut également utiliser la méthode de Monte Carlo lorsque l'on a qu'une seule mesure d'une grandeur cible : c'est le cas notamment en dosage.

Expérience : Des élèves effectuent une préparation de deux solutions en vue d'un dosage. Pour cela, ils disposent d'une solution, notée S_0 , préparée selon le protocole suivant :

On obtient tout d'abord une solution mère S_m par dissolution de $m = 297,0 \text{ mg}$ de bleu patenté V ($M = 582,66 \text{ g.mol}^{-1}$) dans une fiole jaugée de $V_{f1} = 1,0000 \text{ L}$.

On prélève, avec une pipette jaugée, $V_p = 10,00 \text{ mL}$ de solution mère S_m , que l'on dilue dans une fiole jaugée de $V_{f2} = 250,0 \text{ mL}$. On obtient ainsi la solution S_0 .

Calculer la concentration C_m en quantité de matière de la solution mère S_m et C_0 de la solution S_0 .

$$C_m = \frac{m}{M \cdot V_{f1}} = \frac{0,2970}{582,66 \times 1,0000} = 5,097 \times 10^{-4} \text{ mol.L}^{-1}$$

$$C_0 = \frac{C_m \cdot V_p}{V_{f2}} = \frac{5,097 \times 10^{-4} \times 10,00 \times 10^{-3}}{0,2500} = 2,039 \times 10^{-5} \text{ mol.L}^{-1}$$

On veut calculer les incertitudes-type des concentrations C_0 et C_m dont les expressions sont :

$$C_0 = \frac{C_m \cdot V_p}{V_{f2}} \text{ et } C_m = \frac{m}{M \cdot V_{f1}}$$

On constate que C_0 dépend de C_m , le calcul des incertitudes-composés est dans ce cas plus complexe.

La méthode de Monte-Carlo permet d'étudier la variabilité de C_m et C_0 sans avoir besoin d'utiliser des formules de composition d'incertitudes.

Cette variabilité est expliquée par différentes incertitudes qui s'accumulent tout au long du protocole : incertitudes de la pesée, de la masse molaire et de la fiole jaugée pour la dissolution ; incertitudes de la pipette jaugée et de la deuxième fiole jaugée pour la dilution.

On prendra (évaluation des incertitudes-type par une autre approche que statistique - type B) :

- pesée : d'après la notice de la balance, on prendra $u(m) = 1 \text{ mg}$;
- masse molaire : $u(M) = 0,01 \text{ g.mol}^{-1}$ (dernier chiffre significatif, incertitude négligeable) ;
- fiole jaugée : $u(V_{f1}) = 0,0008 \text{ L}$ (à lire sur la fiole jaugée) ;
- pipette jaugée : $u(V_p) = 0,02 \text{ mL}$ (à lire sur la pipette jaugée) ;
- fiole jaugée : $u(V_{f2}) = 0,3 \text{ mL}$ (à lire sur la fiole jaugée).
- burette graduée : $u(V_1) = u(V_2) = 0,05 \text{ mL}$ (à lire sur la burette graduée)

Un jeu de données ($m, M, V_{f1}, V_p, V_{f2}$) est tiré au sort (tirage avec écarts-types connus, loi normale) pour calculer C_m et C_0 . La procédure est répétée un grand nombre de fois. On calcule les moyennes et les écarts-types $u(C_m), u(C_0)$.

Utiliser le programme `INCTYP1` afin de déterminer les valeurs des incertitudes-types.

Objectifs :

- Afficher les histogrammes obtenus ainsi que les valeurs des concentrations et des incertitudes-type
- Ecrire des résultats sous la forme $C = [\bar{C} \pm u(C)] \times 10^{-n}$ avec un chiffre significatif pour les incertitudes-type.

Proposition de programme :

On va utiliser la fonction `np.random.normal()` pour générer des nombres aléatoires depuis une loi normale centrée réduite (ou loi normale standard) en python.

On va ensuite calculer les grandeurs voulues pour chaque couple de nombres aléatoires tirés. La procédure sera répétée 100 000 fois et on affichera les grandeurs et histogrammes demandés

```
import numpy as np
from matplotlib import pyplot
#Renvoie une valeur aléatoire de la variable L[0] d'incertitude-type L[1]
def Alea(L):
    tirage=np.random.normal() #Tirage entre -infini et +infini (loi normale)
    return L[0]+L[1]*tirage
#####
#Entrées
m=[297.e-3,1.e-3]
M=[582.66,0.01]
Vf1=[1.0000,0.0008]
Vp=[10.00e-3,0.02e-3]
Vf2=[250.0e-3,0.3e-3]
#Calcul de Cm et C0
Cm=m[0]/(M[0]*Vf1[0])
C0=Cm*Vp[0]/Vf2[0]
#####
#Méthode de Monte Carlo pour trouver l'incertitude sur Cm et C0
#sans composition des incertitudes
LCm,LC0=[],[]
iteration=100000
for i in range(iteration):
    AleaCm=Alea(m)/(Alea(M)*Alea(Vf1))
```

```

AleaC0=AleaCm*Alea (Vp) /Alea (Vf2)
LCm.append(AleaCm)
LC0.append(AleaC0)
MoyCm=sum(LCm)/iteration
MoyC0=sum(LC0)/iteration
uCm=(1/(iteration-1)*sum((np.array(LCm)-MoyCm)**2.))**0.5
uC0=(1/(iteration-1)*sum((np.array(LC0)-MoyC0)**2.))**0.5
#####
#Affichage
print('Calcul de Cm : ',Cm)
print('Moyenne des Cm : ',MoyCm)
print('u(Cm) : ',uCm)
pyplot.hist(LCm, range = (0.000505, 0.000515), bins = 50, color = 'blue',
edgecolor = 'black')
pyplot.xlabel('Cm (mol/L)')
pyplot.ylabel('effectif')
pyplot.title('Pour 100 000 iterations')
pyplot.show()
print('Calcul de C0 : ',C0)
print('Moyenne des C0 : ',MoyC0)
print('u(C0) : ',uC0)
pyplot.hist(LC0, range = (2.0e-5, 2.1e-5), bins = 50, color = 'blue',
edgecolor = 'black')
pyplot.xlabel('C0 (mol/L)')
pyplot.ylabel('effectif')
pyplot.title('Pour 100 000 iterations')
pyplot.show()

```

OUT:

Calcul de Cm : 0.0005097312326227989

Calcul de C0 : 2.0389249304911954e-05

Moyenne des Cm : 0.0005097315547043968

Moyenne des C0 : 2.0389205938815884e-05

u(Cm) : 1.7624883065844021e-06

u(C0) : 8.479637917001903e-08

